
BAB 4

PRINSIP-PRINSIP PEMODELAN & ANALISIS KEBUTUHAN

Memahami Kebutuhan Pengguna sebagai Fondasi Sistem Berkualitas

Deskripsi Singkat Bab

Bab ini membahas pondasi penting dalam rekayasa perangkat lunak: bagaimana memahami, mendefinisikan, dan memodelkan kebutuhan sistem sebelum memulai pengembangan. Mahasiswa akan mempelajari prinsip-prinsip panduan dalam praktik rekayasa perangkat lunak modern, memahami pentingnya komunikasi dan kolaborasi antara pengembang dan pemangku kepentingan, serta menguasai seluruh proses rekayasa kebutuhan (requirements engineering) mulai dari elisitasi hingga validasi. Bab ini juga memperkenalkan perbedaan antara kebutuhan fungsional dan non-fungsional, serta teknik pemodelan use case yang merupakan alat komunikasi utama antara analis dan pemangku kepentingan. Pembahasan ditutup dengan studi kasus analisis kebutuhan sistem Safe Home sebagaimana diajarkan oleh Pressman (2010).

Referensi utama: Pressman, R.S. (2010). *Software Engineering: A Practitioner's Approach*, Seventh Edition. McGraw-Hill.

4.1 Prinsip-Prinsip Panduan dalam Praktik Rekayasa Perangkat Lunak

Pressman (2010) mengidentifikasi bahwa setiap aktivitas dalam rekayasa perangkat lunak — dari komunikasi hingga deployment — harus dilandasi oleh seperangkat prinsip panduan. Prinsip-prinsip ini bukan aturan yang kaku, melainkan kebijaksanaan yang distilasi dari pengalaman ribuan proyek perangkat lunak selama beberapa dekade.

Memahami prinsip-prinsip ini penting karena mereka memberikan arah ketika praktisi menghadapi situasi yang tidak terduga. Seperti kompas bagi seorang navigator, prinsip-prinsip ini membantu pengembang membuat keputusan yang tepat bahkan dalam kondisi yang ambigu.

4.1.1 Prinsip Komunikasi

Aktivitas komunikasi bertujuan memungkinkan pengembang memahami tujuan pemangku kepentingan dan mengabstraksikan kebutuhan untuk sistem atau produk yang akan dibangun. Pressman merumuskan prinsip-prinsip komunikasi sebagai berikut:

-
1. **Dengarkan sebelum berbicara.** Terlalu banyak orang — terutama insinyur teknis — tidak sabar dalam mendengarkan. Mereka ingin langsung menawarkan solusi sebelum memahami masalah sepenuhnya. Dengarkan secara aktif, bukan pasif.
 2. **Siapkan diri Anda sebelum berkomunikasi.** Lakukan riset dasar tentang domain bisnis klien. Datanglah dengan pengetahuan yang cukup agar komunikasi lebih produktif dan pertanyaan yang diajukan lebih bermakna.
 3. **Seseorang harus memfasilitasi komunikasi.** Dalam setiap pertemuan elicitasi kebutuhan, harus ada seorang fasilitator yang menjaga fokus diskusi, mencegah diskusi melebar, dan memastikan semua pandangan terdengar.
 4. **Komunikasi tatap muka adalah yang terbaik.** Namun, seringkali lebih baik jika ada representasi visual dari topik yang didiskusikan — sketsa, diagram, atau prototipe sederhana di papan tulis.
 5. **Catat dan dokumentasikan segera.** Jangan bergantung pada ingatan. Keputusan, asumsi, dan poin penting yang disepakati dalam pertemuan harus segera didokumentasikan.
 6. **Kolaborasi dan konsensus.** Bekerja sama untuk mencapai pemahaman dan kesepakatan — bukan kompetisi antara pengembang dan klien untuk 'menang' dalam negosiasi.

4.1.2 Prinsip Perencanaan

Perencanaan mendefinisikan pekerjaan rekayasa perangkat lunak dengan mendeskripsikan tugas-tugas teknis yang perlu dilakukan, risiko yang mungkin muncul, sumber daya yang diperlukan, produk kerja yang akan dihasilkan, dan jadwal kerja.

- **Pahami ruang lingkup proyek.** Sebelum membuat rencana, Anda harus memahami dengan jelas apa yang harus dibangun. Ruang lingkup yang tidak jelas adalah penyebab utama perencanaan yang gagal.
- **Rencanakan secara bertahap.** Tidak ada yang dapat merencanakan keseluruhan proyek perangkat lunak besar dengan sempurna di awal. Gunakan perencanaan berlapis: rencana tingkat tinggi untuk keseluruhan, rencana rinci untuk tahap berikutnya.
- **Sesuaikan rencana dengan kebutuhan tim.** Rencana yang dibuat tanpa mempertimbangkan kemampuan dan kapasitas aktual tim tidak akan pernah berhasil.
- **Pertahankan rencana agar tetap relevan.** Ketika keadaan berubah, rencana harus diperbarui. Rencana yang tidak pernah direvisi bukan rencana yang hidup — ia adalah dokumen yang ketinggalan zaman.

4.1.3 Prinsip Pemodelan

Model adalah representasi sesuatu. Dalam rekayasa perangkat lunak, model digunakan untuk merepresentasikan apa yang harus dibangun (model analisis) dan bagaimana cara membangunnya (model desain). Model membantu kita memahami sesuatu yang terlalu kompleks atau abstrak untuk dipahami secara langsung.



Analogi Pemodelan dalam Rekayasa

Seorang arsitek tidak langsung membangun gedung bertingkat. Ia terlebih dahulu membuat:

- Sketsa konseptual untuk mendiskusikan ide dengan klien
- Denah lantai (floor plan) untuk merencanakan tata ruang
- Tampak depan/samping untuk menunjukkan estetika eksterior
- Rencana struktur untuk keperluan konstruksi
- Model 3D digital untuk visualisasi akhir

Setiap model melayani tujuan yang berbeda dan audiens yang berbeda. Prinsip yang sama berlaku dalam rekayasa perangkat lunak: berbagai model (use case, class diagram, sequence diagram) melayani tujuan yang berbeda dalam proses pembangunan sistem.

Prinsip-prinsip pemodelan menurut Pressman:

1. **Tujuan utama model adalah untuk memahami.** Model yang dibuat hanya untuk memenuhi formalitas dokumentasi — tanpa benar-benar digunakan untuk memahami sistem — adalah pemborosan. Setiap model harus memberikan wawasan.
2. **Sesuaikan model dengan sistem yang sedang dibangun.** Berbeda sistem memerlukan model yang berbeda. Sistem real-time memerlukan model yang berbeda dari sistem batch processing.
3. **Pertimbangkan penggunaan beberapa model yang berbeda.** Tidak ada satu model yang dapat menggambarkan semua aspek sistem secara lengkap. Gunakan kombinasi model yang saling melengkapi.
4. **Ketahui kapan harus berhenti.** Tujuan pemodelan adalah pemahaman yang cukup untuk mulai membangun. Pemodelan yang berlebihan (over-modeling) membuang waktu tanpa menambah nilai.

4.1.4 Prinsip Konstruksi

Konstruksi adalah tentang pengkodean dan pengujian — kegiatan yang paling sering diasosiasikan dengan pengembangan perangkat lunak. Prinsip panduan untuk konstruksi mencakup:

- **Pilih pendekatan pemrograman yang tepat.** Pilihan bahasa pemrograman, paradigma (OOP, fungsional, prosedural), dan arsitektur harus didasarkan pada karakteristik masalah, bukan preferensi pribadi.
- **Gunakan standar penulisan kode.** Kode yang konsisten dan mengikuti standar jauh lebih mudah dibaca, dipahami, dan dipelihara oleh orang lain — termasuk oleh diri Anda sendiri 6 bulan ke depan.
- **Pengujian adalah tanggung jawab semua orang.** Pengujian bukan hanya pekerjaan tim QA. Pengembang harus menulis unit test untuk kode mereka sendiri.

4.1.5 Prinsip Deployment

Deployment adalah tentang pengiriman perangkat lunak kepada pengguna. Satu hal yang sering dilupakan adalah bahwa deployment bukan akhir dari proyek — ia adalah awal dari fase operasional yang bisa berlangsung bertahun-tahun.

- **Kelola ekspektasi pengguna.** Seringkali pengguna memiliki ekspektasi yang terlalu tinggi atau terlalu rendah tentang sistem baru. Manajemen ekspektasi yang baik sejak awal akan mengurangi ketidakpuasan saat delivery.
- **Sediakan dukungan yang memadai.** Pengguna pasti akan menghadapi masalah di awal penggunaan. Dokumen panduan, pelatihan, dan helpdesk adalah investasi yang penting.
- **Kumpulkan umpan balik secara sistematis.** Deployment adalah kesempatan emas untuk mendapatkan data nyata tentang bagaimana sistem digunakan dan di mana ia perlu ditingkatkan.

4.2 Komunikasi dan Kolaborasi dalam Rekayasa Kebutuhan

Salah satu penyebab utama kegagalan proyek perangkat lunak adalah kesalahpahaman antara pengembang dan pemangku kepentingan (stakeholders) tentang apa yang sebenarnya harus dibangun. Studi Standish Group menunjukkan bahwa lebih dari 40% cacat (defect) dalam proyek perangkat lunak berasal dari fase requirements — bukan dari fase pengkodean!

Ini menegaskan bahwa rekayasa kebutuhan bukanlah formalitas administratif — ia adalah fondasi yang menentukan apakah sebuah sistem akan berhasil atau gagal. Dan fondasi dari rekayasa kebutuhan adalah komunikasi yang efektif.

4.2.1 Tantangan dalam Komunikasi Kebutuhan

Proses mendapatkan informasi kebutuhan dari pemangku kepentingan ternyata jauh lebih sulit dari yang terlihat. Pressman (2010) mengidentifikasi beberapa hambatan komunikasi yang sering terjadi:

Tantangan	Deskripsi	Contoh
Masalah ruang lingkup (scope)	Klien mendefinisikan terlalu banyak atau terlalu sedikit detail teknis di awal, sering kali mencampuradukkan keinginan dan kebutuhan.	Klien berkata: 'Buat saja aplikasinya bagus dan canggih.' — terlalu samar. Atau: 'Gunakan Python 3.11 dengan arsitektur microservices menggunakan Kafka' — terlalu preskriptif sebelum ada analisis.
Masalah pemahaman (understanding)	Klien tidak sepenuhnya yakin dengan apa yang mereka butuhkan. Mereka tahu ada masalah, tetapi tidak tahu persis solusi yang diinginkan.	'Kami ingin sistem yang bisa meningkatkan produktivitas tim kami' — tanpa menjelaskan bagaimana produktivitas diukur saat ini dan target yang ingin dicapai.
Masalah volatilitas (volatility)	Kebutuhan berubah seiring waktu karena perubahan lingkungan	Sistem manajemen pajak yang dibangun berdasarkan regulasi tahun

Tantangan	Deskripsi	Contoh
	bisnis, regulasi, atau pemahaman yang semakin mendalam tentang masalah.	ini mungkin perlu diubah signifikan ketika peraturan pajak baru diterbitkan tahun depan.
Masalah komunikasi lintas domain	Pengembang berpikir dalam istilah teknis; klien berpikir dalam istilah bisnis. Keduanya sering tidak saling memahami.	Pengembang berkata 'Kita perlu normalisasi database sampai 3NF' — yang sama sekali tidak berarti bagi manajer keuangan yang menginginkan laporan yang akurat.

4.2.2 Teknik Kolaborasi Efektif

Untuk mengatasi tantangan di atas, Pressman (2010) merekomendasikan beberapa teknik kolaborasi yang terbukti efektif dalam elisitasi kebutuhan:

a. Facilitated Application Specification Techniques (FAST)

FAST adalah pendekatan pertemuan terstruktur yang melibatkan pengembang dan pemangku kepentingan untuk bersama-sama mendefinisikan kebutuhan sistem. Prinsip FAST adalah bahwa keduanya adalah mitra yang setara dalam proses ini — bukan klien yang 'memesan' dan vendor yang 'mengerjakan'.

- Semua peserta (pengembang dan stakeholders) duduk bersama dalam satu ruangan.
- Seorang fasilitator netral memimpin diskusi.
- Setiap orang mempersiapkan daftar objek, layanan, batasan, dan kriteria kinerja yang relevan.
- Perbedaan pendapat diselesaikan melalui konsensus, bukan voting atau keputusan sepihak.

b. Quality Function Deployment (QFD)

QFD adalah teknik manajemen kualitas yang diterapkan untuk kebutuhan perangkat lunak. QFD membantu memastikan bahwa kebutuhan pelanggan benar-benar diterjemahkan ke dalam fitur teknis. Teknik ini mengklasifikasikan kebutuhan menjadi tiga kategori:

- **Normal requirements (kebutuhan normal):** Tujuan dan sasaran yang dinyatakan oleh klien selama pertemuan. Misalnya: 'Sistem harus dapat memproses pesanan online.'
- **Expected requirements (kebutuhan tersirat):** Kebutuhan yang begitu mendasar sehingga klien tidak merasa perlu menyebutkannya — tetapi akan sangat kecewa jika tidak ada. Misalnya: 'Sistem harus aman' — tidak disebutkan secara eksplisit, tetapi semua orang mengasumsikannya.
- **Exciting requirements (kebutuhan pemukau):** Fitur-fitur yang melampaui ekspektasi klien dan dapat memberikan kepuasan yang sangat tinggi. Misalnya: rekomendasi produk berbasis AI yang tidak diminta tetapi ternyata sangat disukai pengguna.

c. Penggunaan Scenario (Skenario Penggunaan)

Skenario adalah deskripsi tekstual tentang bagaimana seseorang berinteraksi dengan sebuah sistem dalam konteks tertentu. Skenario adalah cara yang paling alami bagi manusia untuk menggambarkan kebutuhan — melalui narasi cerita, bukan daftar spesifikasi teknis yang kaku.



Contoh Skenario: Sistem Pemesanan Makanan Online

Nama Skenario: Memesan Makanan untuk Dine-In

Pelaku utama: Pelanggan (Rina, 25 tahun, mahasiswi)

Konteks: Rina duduk di meja nomor 7 di restoran. Ia melihat QR code di meja dan memindainya menggunakan kamera ponselnya.

Alur normal:

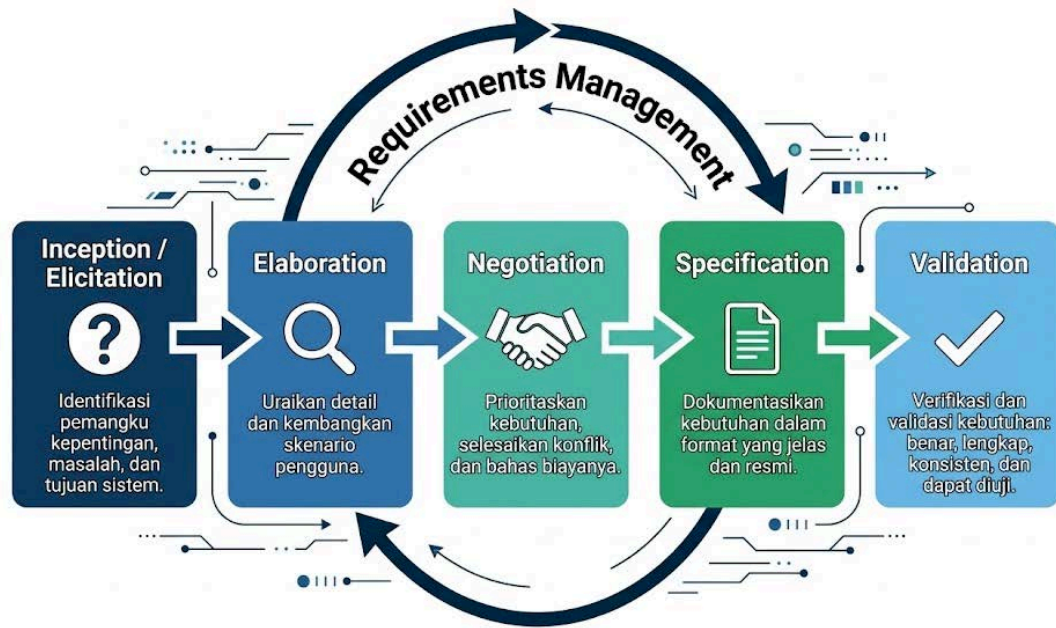
1. Rina memindai QR code. Sistem menampilkan menu digital restoran.
2. Rina menelusuri menu dan memilih 'Nasi Goreng Spesial' dan 'Es Teh Manis'.
3. Sistem menampilkan ringkasan pesanan dan total harga.
4. Rina menekan 'Konfirmasi Pesanan'.
5. Sistem mengirimkan pesanan ke dapur dan menampilkan estimasi waktu tunggu (15 menit).
6. Rina menerima notifikasi ketika pesanan siap diantar.

Kondisi yang mungkin:

- Jika item habis: sistem menampilkan pesan 'Stok habis' dan menyarankan menu alternatif.
- Jika koneksi internet terputus: sistem menyimpan pesanan sementara dan mengirimkan ketika koneksi kembali.

4.3 Requirements Engineering: Dari Kebutuhan ke Spesifikasi

Requirements Engineering (Rekayasa Kebutuhan) adalah kumpulan kegiatan yang membantu pengembang memahami, mendokumentasikan, dan memvalidasi kebutuhan sistem. Pressman (2010) mendefinisikan requirements engineering sebagai spektrum kegiatan yang memungkinkan tim perangkat lunak untuk mendefinisikan kebutuhan sistem atau produk yang akan dibangun.



Gambar 1: Lima Kegiatan Requirements Engineering

Diagram ini menunjukkan bahwa requirements engineering bukan proses linear yang kaku — panah dua arah antar kotak menegaskan bahwa pengembang sering perlu 'mundur' ke kegiatan sebelumnya ketika menemukan informasi baru. Misalnya, saat elaborasi muncul pertanyaan baru yang mengharuskan kembali ke elicitation.

4.3.1 Inception dan Elicitation (Pencarian Kebutuhan)

Inception adalah titik di mana kebutuhan sebuah sistem pertama kali diidentifikasi. Ini adalah pertemuan awal antara pengembang dan pemangku kepentingan untuk memahami: Apa masalah yang ingin diselesaikan? Siapa yang akan menggunakan sistem? Mengapa sistem ini dibutuhkan?

Elicitation (elitisasi) adalah proses aktif untuk mengumpulkan informasi kebutuhan dari berbagai sumber. Kata 'elicitation' sengaja dipilih — bukan 'collection' (pengumpulan) — karena ini bukan proses pasif menerima informasi yang sudah ada, melainkan proses aktif 'menggali' informasi yang tersembunyi atau belum terformulasi.

Teknik-teknik elitisasi yang umum digunakan:

- **Wawancara (Interview):** Percakapan terstruktur atau semi-terstruktur dengan pemangku kepentingan kunci. Cocok untuk mendapatkan pemahaman mendalam tentang perspektif individual. Pertanyaan terbuka ("Ceritakan bagaimana proses X berjalan saat ini?") lebih efektif daripada pertanyaan tertutup ("Apakah Anda butuh fitur Y?").
- **Kuesioner (Questionnaire):** Survei tertulis yang cocok untuk menjangkau banyak pemangku kepentingan dalam waktu singkat. Berguna untuk mengumpulkan data kuantitatif dan mengidentifikasi pola.
- **Observasi (Observation):** Analisis mengamati langsung pengguna dalam menjalankan pekerjaan mereka sehari-hari. Sangat efektif untuk menemukan

kebutuhan yang tidak terucapkan (tacit requirements) — kebutuhan yang pengguna tidak sadari atau tidak bisa formulasikan karena sudah menjadi rutinitas.

- **Analisis Dokumen:** Mempelajari dokumen yang sudah ada: prosedur operasional, formulir, laporan, buku panduan. Berguna untuk memahami proses bisnis saat ini.
- **Workshop / Focus Group:** Pertemuan kelompok terstruktur di mana beberapa pemangku kepentingan berkumpul bersama untuk mendefinisikan kebutuhan. Sangat efektif untuk mendapatkan konsensus dan menangkap interaksi antar pemangku kepentingan.
- **Prototyping:** Membangun model awal (mockup) sistem untuk membantu pemangku kepentingan memvisualisasikan sistem dan mengklarifikasi kebutuhan mereka. Sangat efektif ketika pengguna kesulitan mengekspresikan kebutuhan secara verbal.

4.3.2 Elaboration (Pendalaman Kebutuhan)

Informasi yang dikumpulkan dalam elisitasi sering kali masih kasar, ambigu, atau tidak lengkap. Elaborasi adalah proses menganalisis, memperluas, dan memperhalus informasi tersebut menjadi representasi yang lebih terstruktur dan dapat dianalisis secara teknis.

Pada tahap elaborasi, analis biasanya membuat model awal dari sistem — bukan model final, melainkan model eksplorasi yang membantu mengidentifikasi kesenjangan dan inkonsistensi dalam informasi kebutuhan.

Contoh Elaborasi: Dari Pernyataan Kasar ke Kebutuhan Terstruktur

Pernyataan elicitasi (dari wawancara dengan manajer toko):

"Kami ingin sistem yang bisa tahu kapan stok barang mau habis dan langsung pesan ke supplier."

Hasil elaborasi:

Objek yang diidentifikasi: Produk, StokBarang, Supplier, PesananPengadaan, AlertStok

Kebutuhan yang diekstrak:

- Sistem harus memantau kuantitas stok untuk setiap produk secara real-time.
- Sistem harus membandingkan stok saat ini dengan reorder point yang telah ditetapkan.
- Ketika stok mencapai atau di bawah reorder point, sistem harus membuat draft pesanan pengadaan otomatis.
- Draft pesanan harus dikirimkan ke manajer pembelian untuk disetujui sebelum dikirim ke supplier.
- Sistem harus memiliki daftar supplier aktif beserta produk yang mereka suplai.

Pertanyaan yang muncul (untuk clarification):

- Apa yang dimaksud dengan 'mau habis'? Bagaimana reorder point ditetapkan?
 - Apakah pesanan ke supplier langsung dikirim secara otomatis atau perlu approval dulu?
 - Bagaimana jika ada beberapa supplier untuk satu produk — mana yang dipilih?
-

4.3.3 Negotiation (Negosiasi Kebutuhan)

Dalam proyek nyata, sangat jarang semua pemangku kepentingan memiliki kebutuhan yang sepenuhnya selaras. Negosiasi adalah proses menemukan kesepakatan di antara berbagai pihak yang memiliki kepentingan berbeda, prioritas berbeda, atau bahkan kebutuhan yang saling bertentangan.

Negosiasi yang efektif bukan tentang siapa yang 'menang' atau 'kalah', melainkan tentang menemukan solusi terbaik yang memberikan nilai maksimum dalam batasan yang ada (waktu, biaya, sumber daya).

Sumber Konflik	Contoh	Pendekatan Resolusi
Prioritas berbeda antar stakeholder	Manajer penjualan menginginkan fitur laporan real-time, manajer IT menginginkan keamanan data diprioritaskan — keduanya tidak bisa dikerjakan bersamaan dalam satu sprint	Gunakan teknik MoSCoW (Must Have, Should Have, Could Have, Won't Have) untuk memprioritaskan bersama
Anggaran vs fitur	Klien menginginkan 50 fitur, tetapi anggaran hanya cukup untuk 20 fitur	Buat matriks nilai-biaya: identifikasi 20 fitur dengan nilai bisnis tertinggi dan biaya pengembangan terendah
Kebutuhan teknis vs bisnis	Tim teknis ingin migrasi ke arsitektur microservices, tim bisnis ingin aplikasi rilis dalam 3 bulan — migrasi butuh 6 bulan	Diskusikan trade-off secara transparan dan cari solusi tengah: rilis dengan arsitektur monolith yang modular, rencanakan migrasi bertahap
Kebutuhan yang bertentangan	Tim A membutuhkan antarmuka yang sangat sederhana; tim B membutuhkan antarmuka yang sangat detail dengan banyak opsi	Pertimbangkan dua mode tampilan (basic dan advanced) sebagai kompromi

4.3.4 Specification (Spesifikasi Kebutuhan)

Specification adalah proses mendokumentasikan kebutuhan yang telah disepakati ke dalam format yang resmi dan dapat dirujuk oleh semua pihak. Hasil dari kegiatan ini adalah dokumen Software Requirements Specification (SRS).

Dokumen SRS yang baik harus memiliki karakteristik berikut:

- **Benar (Correct):** Setiap kebutuhan yang dinyatakan merepresentasikan sesuatu yang dibutuhkan sistem.
 - **Tidak ambigu (Unambiguous):** Setiap kebutuhan hanya memiliki satu interpretasi. Hindari kata-kata seperti 'cepat', 'mudah', 'efisien' tanpa definisi yang terukur.
-

-
- **Lengkap (Complete):** Dokumen mencakup semua kondisi yang signifikan: normal, alternatif, dan kondisi kesalahan.
 - **Konsisten (Consistent):** Tidak ada kebutuhan yang saling bertentangan satu sama lain.
 - **Dapat diverifikasi (Verifiable):** Setiap kebutuhan dapat diuji atau diverifikasi untuk memastikan sistem telah memenuhinya.
 - **Dapat ditelusuri (Traceable):** Setiap kebutuhan dapat dikaitkan kembali ke sumbernya (wawancara dengan siapa, tanggal berapa) dan ke depan ke komponen sistem yang mengimplementasikannya.

⚠ **Contoh Kebutuhan yang Ambiguous vs. Tidak Ambiguous**

AMBIGUOUS (hindari):

"Sistem harus memproses pesanan dengan cepat."

→ Seberapa cepat? Satu detik? Satu menit? Tidak ada cara untuk menguji apakah sistem sudah memenuhi syarat ini.

TIDAK AMBIGUOUS (lebih baik):

"Sistem harus memproses dan menampilkan konfirmasi pesanan dalam waktu kurang dari 3 detik untuk 95% transaksi pada kondisi beban normal (maksimal 100 pengguna konkuren)."

→ Jelas, terukur, dan dapat diuji. Ini adalah spesifikasi yang baik.

4.3.5 Validation (Validasi Kebutuhan)

Validasi adalah proses memeriksa apakah kebutuhan yang telah dispesifikasikan benar-benar merepresentasikan sistem yang diinginkan oleh pemangku kepentingan. Ini adalah 'pemeriksaan terakhir' sebelum kebutuhan digunakan sebagai dasar untuk desain dan pengembangan.

Perbedaan penting yang perlu dipahami:

- **Validasi:** Apakah kita membangun sistem yang BENAR? (Apakah spesifikasi ini mencerminkan apa yang diinginkan pengguna?)
- **Verifikasi:** Apakah kita membangun sistem dengan CARA YANG BENAR? (Apakah sistem yang dibangun sesuai dengan spesifikasi?)

Teknik validasi kebutuhan yang umum digunakan:

1. **Requirements Review:** Tim pengembang dan pemangku kepentingan secara bersama-sama meninjau dokumen kebutuhan untuk menemukan kesalahan, inkonsistensi, atau hal yang terlewat.
 2. **Prototyping:** Membangun prototipe sistem berdasarkan kebutuhan yang dispesifikasikan, lalu meminta pengguna menggunakannya untuk memverifikasi apakah prototipe sesuai dengan kebutuhan mereka.
-

-
3. **Test Case Generation:** Mencoba menulis test case berdasarkan spesifikasi kebutuhan. Jika sebuah kebutuhan sulit atau tidak mungkin diuji, itu adalah tanda bahwa kebutuhan tersebut tidak cukup jelas atau tidak terukur.
 4. **Automated Consistency Analysis:** Menggunakan tools untuk memeriksa konsistensi dan kelengkapan dokumen kebutuhan secara otomatis.

4.4 Functional vs Non-Functional Requirements

Salah satu klasifikasi paling fundamental dalam rekayasa kebutuhan adalah membedakan antara kebutuhan fungsional dan kebutuhan non-fungsional. Pemahaman yang baik tentang perbedaan ini sangat penting karena keduanya ditangani dengan cara yang berbeda dalam proses desain dan pengujian.

4.4.1 Functional Requirements (Kebutuhan Fungsional)

Kebutuhan fungsional mendeskripsikan fungsi-fungsi spesifik yang harus dapat dilakukan sistem — apa yang harus dilakukan sistem dalam respons terhadap input atau kondisi tertentu. Kebutuhan fungsional mendefinisikan perilaku sistem yang dapat diamati.

Cara termudah untuk mengidentifikasi kebutuhan fungsional adalah dengan bertanya: "Apa yang sistem harus lakukan?" atau "Apa yang pengguna ingin lakukan dengan sistem?"

Contoh Kebutuhan Fungsional: Aplikasi E-Learning

FR-01: Sistem harus mengizinkan instruktur untuk mengunggah materi kuliah dalam format PDF, PPT, dan video MP4 dengan ukuran maksimum 500 MB per file.

FR-02: Sistem harus menampilkan progress belajar mahasiswa dalam bentuk persentase penyelesaian modul.

FR-03: Sistem harus mengirimkan notifikasi email kepada mahasiswa 24 jam sebelum deadline pengumpulan tugas.

FR-04: Sistem harus memungkinkan instruktur membuat kuis dengan tipe soal: pilihan ganda, benar/salah, dan essay.

FR-05: Sistem harus menghitung nilai akhir mahasiswa berdasarkan bobot komponen: kuis (30%), tugas (30%), ujian akhir (40%).

FR-06: Sistem harus menghasilkan sertifikat kelulusan secara otomatis bagi mahasiswa yang menyelesaikan kursus dengan nilai minimum 70.

4.4.2 Non-Functional Requirements (Kebutuhan Non-Functional)

Kebutuhan non-fungsional mendeskripsikan batasan dan kualitas yang harus dipenuhi sistem — bukan apa yang sistem lakukan, tetapi bagaimana cara sistem melakukannya. Kebutuhan non-fungsional sering disebut juga sebagai quality attributes (atribut kualitas) atau system constraints (batasan sistem).

Cara mengidentifikasi kebutuhan non-fungsional: "Seberapa baik, seberapa cepat, seberapa aman, seberapa andal sistem harus melakukannya?"



Gambar 2: Kategori Non-Functional Requirements

Diagram roda ini memperlihatkan bahwa non-functional requirements bukan satu hal tunggal, melainkan sekumpulan dimensi kualitas yang berbeda-beda. Dalam proyek nyata, tidak semua dimensi sama pentingnya — misalnya, sistem perbankan akan sangat memprioritaskan Security dan Reliability, sementara aplikasi media sosial akan lebih memprioritaskan Performance dan Scalability.

Kategori NFR	Deskripsi	Contoh Konkret
Performance	Seberapa cepat sistem merespons dan seberapa banyak beban yang dapat ditangani.	Halaman beranda harus dimuat dalam < 2 detik pada koneksi 3G. Sistem harus mendukung 10.000 pengguna konkuren.
Security	Kemampuan sistem melindungi data dan fungsinya dari akses yang tidak sah.	Semua komunikasi harus dienkripsi dengan TLS 1.3. Password harus di-hash menggunakan bcrypt. Sistem harus memblokir akun setelah 5 kali gagal login.
Reliability	Seberapa andal sistem beroperasi tanpa kegagalan.	Sistem harus mencapai uptime 99,9% (downtime maksimal 8,7 jam/tahun). Waktu pemulihan dari kegagalan tidak boleh lebih dari 30 menit.
Usability	Seberapa mudah sistem digunakan oleh pengguna target.	Pengguna baru harus dapat menyelesaikan tugas utama (membuat pesanan) tanpa pelatihan, dalam waktu kurang dari 5 menit.
Scalability	Kemampuan sistem untuk berkembang seiring meningkatnya beban.	Sistem harus dapat diskalakan secara horizontal untuk menangani lonjakan trafik hingga 10x lipat tanpa perubahan arsitektur.

Kategori NFR	Deskripsi	Contoh Konkret
Maintainability	Seberapa mudah sistem dipelihara dan dimodifikasi.	Kode harus memenuhi standar kualitas SonarQube dengan maximum technical debt 5%. Dokumentasi API harus selalu diperbarui.

⚠ Mengapa NFR Sering Diabaikan dan Akibatnya

Kebutuhan non-fungsional sering diabaikan karena tidak terlihat secara langsung oleh pengguna — sampai sistem gagal memenuhinya.

Contoh kasus nyata: Sebuah marketplace online baru berhasil membangun semua fitur yang diminta (kebutuhan fungsional terpenuhi). Namun saat diluncurkan ke publik dan mendapat 10.000 pengguna dalam satu hari, sistem langsung down karena tidak dirancang untuk menangani beban tersebut (kebutuhan scalability dan performance tidak dispesifikasikan sejak awal).

Pelajaran: Kegagalan memenuhi NFR dapat membuat sistem yang fungsionalitasnya lengkap sekalipun menjadi tidak berguna atau bahkan berbahaya dalam kondisi nyata.

4.5 Use Case: Skenario, Aktor, dan Use Case Diagram

Use case adalah teknik pemodelan yang dikembangkan oleh Ivar Jacobson pada tahun 1992 dan kemudian menjadi bagian dari standar UML (Unified Modeling Language). Use case mendeskripsikan cara pengguna berinteraksi dengan sistem untuk mencapai tujuan tertentu.

Use case adalah alat komunikasi yang sangat efektif karena ditulis dalam bahasa yang dapat dipahami oleh pemangku kepentingan non-teknis sekaligus cukup terstruktur untuk menjadi panduan bagi pengembang. Ia menjembatani dunia bisnis dan dunia teknis.

4.5.1 Aktor (Actor)

Aktor adalah entitas yang berinteraksi dengan sistem dari luar. Aktor bisa berupa:

- **Pengguna manusia:** Orang yang langsung berinteraksi dengan sistem melalui antarmuka. Contoh: Pelanggan, Administrator, Kasir, Dokter.
- **Sistem eksternal:** Sistem lain yang berinteraksi dengan sistem yang sedang dikembangkan. Contoh: Payment Gateway, Email Server, Sistem Bank.
- **Perangkat keras:** Sensor atau perangkat fisik yang memberikan input ke sistem. Contoh: Sensor gerak, kamera CCTV, barcode scanner.

Penting untuk dipahami bahwa aktor bukan orang atau sistem tertentu, melainkan peran yang dimainkan. Satu orang fisik bisa memainkan beberapa peran (aktor). Misalnya, seorang staf kasir bisa berperan sebagai 'Kasir' ketika memproses transaksi, dan berperan sebagai 'Manajer Shift' ketika membuka atau menutup kas.

4.5.2 Use Case

Sebuah use case mendeskripsikan urutan aksi yang dilakukan sistem untuk menghasilkan sesuatu yang bernilai bagi aktor. Setiap use case memiliki:

- **Nama use case:** Ditulis dengan kata kerja + objek yang mencerminkan tujuan pengguna. Contoh: 'Melakukan Pemesanan', 'Mendaftar Akun', 'Mencetak Laporan'.
- **Aktor yang terlibat:** Siapa yang memulai use case (primary actor) dan siapa yang berpartisipasi (secondary actor).
- **Prasyarat (Precondition):** Kondisi yang harus terpenuhi sebelum use case dapat dimulai. Contoh: 'Pengguna harus sudah login'.
- **Alur normal (Main flow):** Langkah-langkah sukses utama dari use case.
- **Alur alternatif (Alternative flow):** Variasi atau kondisi khusus yang menyimpang dari alur normal tetapi masih menghasilkan output yang valid.
- **Alur kesalahan (Exception flow):** Kondisi kesalahan atau kegagalan yang mungkin terjadi.
- **Pasca-kondisi (Postcondition):** Kondisi sistem setelah use case berhasil atau gagal dieksekusi.

Contoh Use Case Lengkap: Melakukan Pembayaran

Nama Use Case: Melakukan Pembayaran

Aktor Utama: Pelanggan

Aktor Pendukung: Payment Gateway

Prasyarat: Pelanggan sudah login dan keranjang belanja tidak kosong.

ALUR NORMAL:

1. Pelanggan membuka halaman checkout.
2. Sistem menampilkan ringkasan pesanan: daftar item, subtotal, ongkos kirim, dan total.
3. Pelanggan memilih metode pembayaran (Transfer Bank / e-Wallet / Kartu Kredit).
4. Pelanggan menekan tombol 'Bayar Sekarang'.
5. Sistem mengirimkan permintaan transaksi ke Payment Gateway.
6. Payment Gateway memproses transaksi dan mengembalikan status.
7. Sistem menerima konfirmasi pembayaran berhasil.
8. Sistem membuat nomor pesanan, memperbarui status pesanan menjadi 'Dibayar', dan mengirimkan email konfirmasi ke pelanggan.
9. Sistem menampilkan halaman 'Pembayaran Berhasil' dengan nomor pesanan.

ALUR ALTERNATIF: Metode Bayar Nanti (COD)

- 3a. Pelanggan memilih metode COD (Cash on Delivery).
- 3b. Sistem melewati langkah 5-7 dan langsung membuat nomor pesanan.

ALUR KESALAHAN: Pembayaran Gagal

- 7a. Payment Gateway mengembalikan status gagal (saldo tidak cukup, kartu ditolak).
 - 7b. Sistem menampilkan pesan kesalahan yang informatif.
-

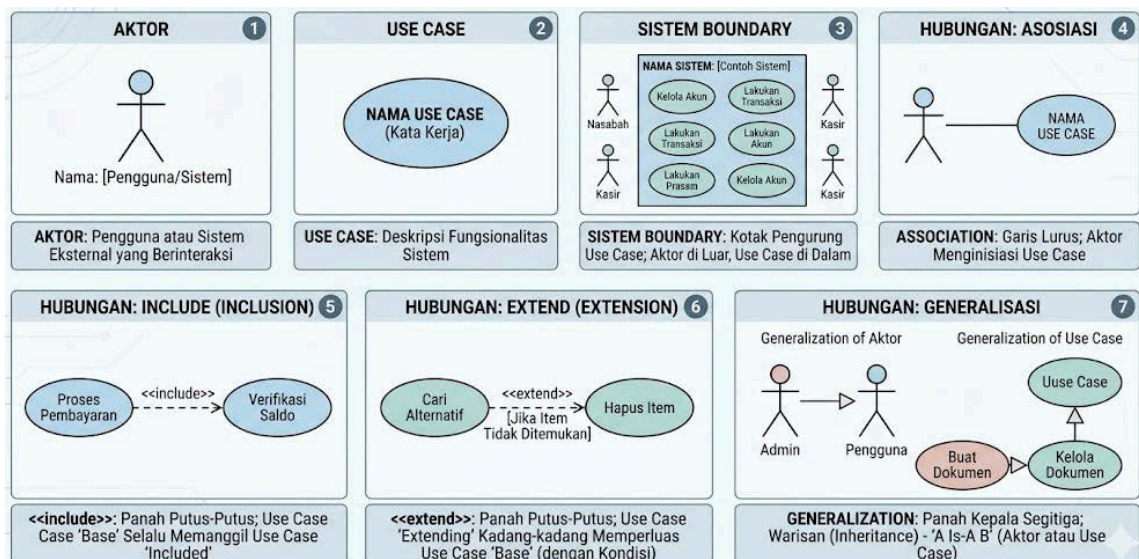
7c. Sistem menawarkan pilihan: coba metode pembayaran lain atau simpan pesanan sebagai draft.

Pasca-kondisi (Sukses): Pesanan tersimpan di database dengan status 'Dibayar', stok produk dikurangi.

Pasca-kondisi (Gagal): Pesanan tersimpan dengan status 'Menunggu Pembayaran', stok tidak berubah.

4.5.3 Use Case Diagram

Use case diagram adalah representasi visual dari use case dan aktor beserta hubungan di antara mereka. Diagram ini memberikan gambaran tingkat tinggi tentang fungsionalitas sistem dari perspektif pengguna.



Gambar 3: Elemen-Elemen Use Case Diagram (Notasi UML)

Memahami elemen-elemen ini secara terpisah terlebih dahulu sangat penting sebelum membaca atau membuat use case diagram yang lebih kompleks. Banyak mahasiswa yang membuat kesalahan dalam membedakan <<include>> dan <<extend>> — ilustrasi ini membantu mengklarifikasi perbedaannya.

💡 Perbedaan <<include>> dan <<extend>>

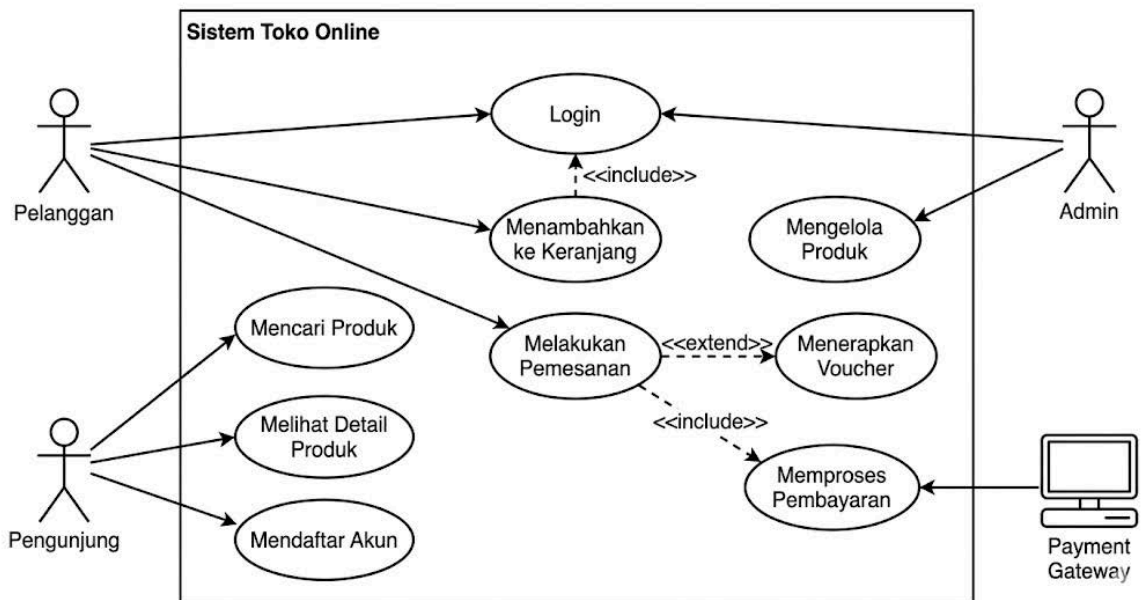
<<include>>: Use case A SELALU memanggil use case B setiap kali A dieksekusi.

Contoh: 'Melakukan Pemesanan' selalu <<include>> 'Autentikasi Pengguna' — kamu tidak bisa memesan tanpa login.

<<extend>>: Use case B KADANG-KADANG memperluas use case A, hanya dalam kondisi tertentu.

Contoh: 'Melakukan Pemesanan' kadang <<extend>> 'Menerapkan Kode Voucher' — tidak semua pelanggan menggunakan voucher, hanya mereka yang punya.

Cara mudah mengingat: <<include>> = wajib/selalu; <<extend>> = opsional/kondisional



Gambar 4: Contoh Use Case Diagram: Sistem Toko Online

Diagram ini menunjukkan bagaimana use case diagram dapat menggambarkan seluruh fungsionalitas utama sebuah sistem dalam satu gambar yang mudah dipahami oleh semua pemangku kepentingan. Perhatikan bagaimana aktor 'Pelanggan' dan 'Pengunjung' memiliki akses ke use case yang berbeda — ini mencerminkan perbedaan hak akses yang harus diimplementasikan dalam sistem.

4.6 Studi Kasus: Analisis Kebutuhan Sistem Safe Home

Sistem Safe Home adalah contoh kasus yang digunakan secara konsisten oleh Pressman (2010) sepanjang bukunya untuk mendemonstrasikan berbagai konsep rekayasa perangkat lunak. SafeHome adalah sistem keamanan rumah cerdas yang memungkinkan pemilik rumah memantau dan mengontrol berbagai aspek keamanan dan otomasi rumah mereka.

Deskripsi Sistem SafeHome

SafeHome adalah produk yang memungkinkan pemilik rumah untuk:

- Memantau dan mengontrol sensor keamanan (sensor gerak, sensor pintu/jendela, sensor asap).
- Mengontrol perangkat rumah dari jarak jauh melalui smartphone (kunci pintu, lampu, thermostat).
- Menerima notifikasi alarm saat terjadi kondisi yang mencurigakan atau berbahaya.
- Berkomunikasi dengan layanan monitoring keamanan 24 jam.
- Mengelola kode akses untuk anggota keluarga dan tamu.

Pengguna utama sistem: Pemilik Rumah (Homeowner)

Sistem eksternal: Central Monitoring Station, Internet Service Provider, Smartphone App

4.6.1 Elicitation: Transkrip Wawancara Awal

Fragmen Wawancara: Analisis dengan Calon Pengguna SafeHome (Pressman, 2010)

Analisis: "Bisakah Anda mendeskripsikan secara umum bagaimana sistem harus bekerja?"

Pelanggan: "Kami ingin menggunakan sistem Safe Home untuk melindungi rumah dan keluarga kami. Sistem harus bisa mendeteksi intruder (pencuri), juga kebakaran atau banjir... dan sistem harus bisa diakses dari manapun melalui internet."

Analisis: "Oke, baik. Bisa Anda ceritakan lebih detail tentang bagaimana Anda ingin sistem mendeteksi intruder?"

Pelanggan: "Kami ingin pasang sensor di semua pintu dan jendela. Kalau ada yang buka saat sistem alarm aktif, sistem harus langsung berbunyi. Dan kirim notifikasi ke ponsel saya. Oh ya, ada kamera juga yang bisa saya lihat dari ponsel."

Analisis: "Bagaimana dengan kode akses? Siapa yang boleh masuk dan keluar?"

Pelanggan: "Tentu saja saya dan istri saya. Dan anak-anak yang sudah cukup besar. Kami ingin bisa atur kode masing-masing orang, jadi kalau ada tamu yang datang saat kami tidak di rumah, bisa kami beri kode sementara."

Analisis: "Seberapa cepat sistem harus bereaksi setelah sensor terpicu?"

Pelanggan: "Secepat mungkin. Tidak boleh lebih dari... katakanlah 10 detik?"

4.6.2 Elaborasi Kebutuhan SafeHome

Berdasarkan wawancara di atas, analisis melakukan elaborasi dan mengidentifikasi elemen-elemen kebutuhan berikut:

Objek-Objek yang Diidentifikasi:

- Sensor (Gerak, Pintu, Jendela, Asap, Kelembapan)
- Camera (indoor, outdoor)
- Panel Kontrol (control panel) — antarmuka fisik di rumah
- Pengguna (Pemilik Rumah, Anggota Keluarga, Tamu)
- Kode Akses
- Alarm (audio, visual, notifikasi digital)
- Pusat Monitoring (Central Monitoring Station)
- Aplikasi Mobile (SafeHome App)

Layanan-Layanan yang Diidentifikasi:

- Memantau status semua sensor secara real-time
- Mengaktifkan/menonaktifkan sistem alarm
- Memproses trigger alarm ketika sensor terpicu
- Mengirimkan notifikasi kepada pengguna dan pusat monitoring
- Mengelola kode akses (tambah, ubah, hapus, atur masa berlaku)
- Streaming video dari kamera
- Mencatat log semua kejadian (event log)

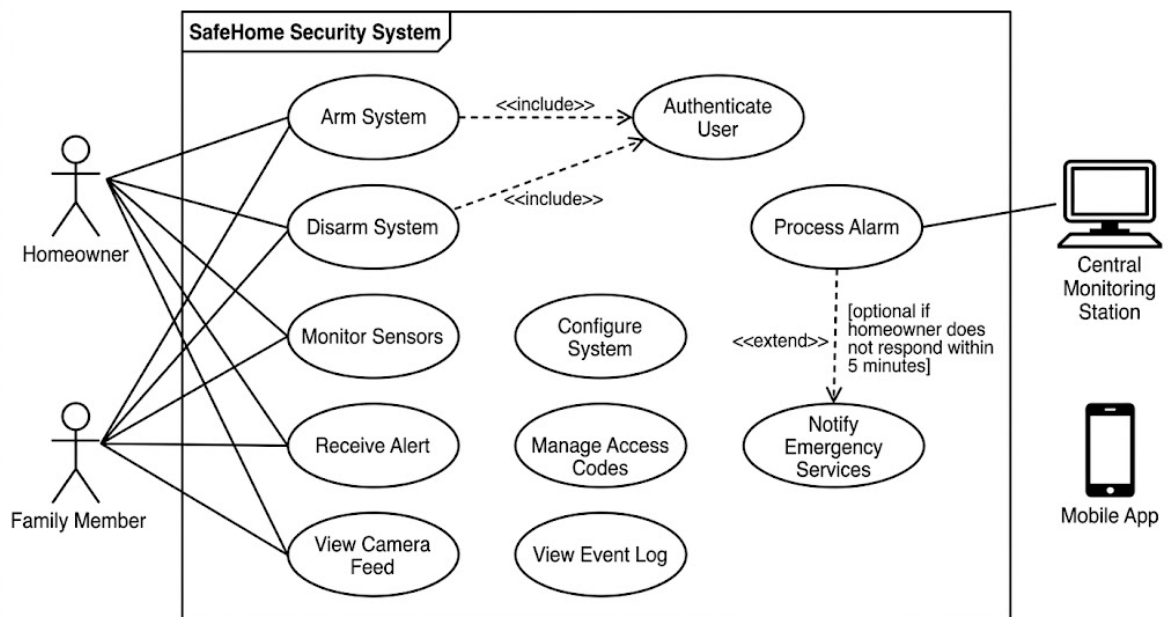
4.6.3 Spesifikasi Kebutuhan SafeHome

Berikut adalah sebagian dari spesifikasi kebutuhan sistem Safe Home:

ID	Tipe	Pernyataan Kebutuhan	Prioritas
SH-FR-01	Fungsional	Sistem harus dapat mendeteksi pelanggaran sensor (sensor door, window, motion) dan mengaktifkan alarm dalam waktu ≤ 8 detik.	Tinggi

ID	Tipe	Pernyataan Kebutuhan	Prioritas
SH-FR-02	Fungsional	Sistem harus mengirimkan notifikasi push ke semua perangkat terdaftar milik pemilik rumah dalam waktu ≤ 15 detik setelah alarm aktif.	Tinggi
SH-FR-03	Fungsional	Sistem harus mengizinkan pemilik rumah untuk membuat, mengubah, dan menghapus kode akses melalui aplikasi mobile.	Tinggi
SH-FR-04	Fungsional	Sistem harus menyediakan streaming video live dari kamera yang terpasang dengan delay maksimal 3 detik.	Sedang
SH-FR-05	Fungsional	Sistem harus menyimpan event log seluruh aktivitas sensor, alarm, dan akses pengguna selama 90 hari terakhir.	Sedang
SH-FR-06	Fungsional	Sistem harus dapat berkomunikasi dengan pusat monitoring eksternal menggunakan protokol yang kompatibel.	Tinggi
SH-NFR-01	Non-Fungsional (Reliability)	Sistem harus beroperasi selama 99,99% waktu (downtime maks. 52 menit/tahun), termasuk saat koneksi internet terputus melalui backup GSM.	Kritis
SH-NFR-02	Non-Fungsional (Security)	Seluruh komunikasi antara panel kontrol, aplikasi mobile, dan server harus dienkripsi end-to-end menggunakan AES-256.	Kritis
SH-NFR-03	Non-Fungsional (Usability)	Pemilik rumah tanpa latar belakang teknis harus dapat mengoperasikan fungsi utama sistem (arm, disarm, cek status) dalam waktu ≤ 30 detik.	Tinggi
SH-NFR-04	Non-Fungsional (Performance)	Aplikasi mobile harus dapat menampilkan status semua sensor dalam waktu ≤ 2 detik pada koneksi 4G.	Tinggi

4.6.4 Use Case Diagram SafeHome



Gambar 5: Use Case Diagram Sistem SafeHome

Use case diagram ini memberikan gambaran lengkap tentang siapa yang dapat melakukan apa dalam sistem Safe Home. Perhatikan bagaimana 'Family Member' memiliki akses terbatas dibandingkan 'Homeowner' — hanya dapat arm/disarm dan menerima notifikasi, tetapi tidak bisa mengkonfigurasi sistem atau mengelola kode akses.

4.6.5 Skenario Lengkap: Homeowner Mengaktifkan Sistem Alarm

Use Case: Arm System (Mengaktifkan Sistem Alarm)

ID Use Case: UC-01

Nama: Arm System

Aktor Utama: Homeowner / Family Member

Prasyarat: Pengguna sudah login melalui aplikasi mobile atau panel kontrol fisik.

ALUR NORMAL:

1. Pengguna membuka aplikasi Safe Home atau mendekati panel kontrol.
2. Sistem menampilkan status sistem saat ini (DISARMED) dan status semua sensor.
3. Pengguna memilih mode alarm: 'Away Mode' (semua sensor aktif) atau 'Stay Mode' (hanya perimeter, interior nonaktif).
4. Pengguna menekan tombol 'Arm' dan memasukkan PIN.
5. Sistem memvalidasi PIN.
6. Sistem memberikan countdown 30 detik (exit delay) untuk memungkinkan pengguna keluar rumah.
7. Setelah countdown, sistem mengaktifkan semua sensor sesuai mode yang dipilih.

8. Sistem menampilkan status 'ARMED — [Mode]' dan mengirimkan notifikasi ke semua perangkat terdaftar.

ALUR ALTERNATIF A: Sensor dalam kondisi fault

3a. Sistem mendeteksi ada sensor yang fault (tidak merespons).

3b. Sistem menampilkan daftar sensor bermasalah.

3c. Pengguna dapat memilih: batalkan arm, atau abaikan sensor bermasalah dan lanjutkan arm.

ALUR KESALAHAN: PIN Salah

5a. PIN tidak valid.

5b. Sistem menampilkan pesan kesalahan dan meminta PIN ulang.

5c. Setelah 3 kali gagal, sistem mengunci panel selama 5 menit dan mengirimkan notifikasi keamanan.

Pasca-kondisi (Sukses): Sistem dalam status ARMED, semua sensor aktif memantau.

Pasca-kondisi (Gagal): Sistem tetap DISARMED, kejadian dicatat dalam event log.

Ringkasan Bab

Bab ini telah membahas pondasi penting dalam rekayasa perangkat lunak — bagaimana memahami dan memodelkan kebutuhan sistem secara sistematis. Berikut poin-poin kunci yang perlu diingat:

- **Prinsip-prinsip panduan** dalam rekayasa perangkat lunak (komunikasi, perencanaan, pemodelan, konstruksi, deployment) memberikan arah bagi praktisi dalam membuat keputusan yang tepat di setiap tahap pengembangan.
 - **Komunikasi dan kolaborasi** adalah fondasi dari rekayasa kebutuhan. Lebih dari 40% cacat perangkat lunak berasal dari fase requirements. Teknik seperti FAST, QFD, dan scenario-based elicitation membantu mengatasi hambatan komunikasi antara pengembang dan pemangku kepentingan.
 - **Requirements Engineering** mencakup lima kegiatan yang saling terkait: Elicitation (menggali), Elaboration (mendalami), Negotiation (menyepakati), Specification (mendokumentasikan), dan Validation (memverifikasi). Prosesnya bersifat iteratif, bukan linear.
 - **Functional requirements** mendefinisikan apa yang sistem lakukan, sedangkan non-functional requirements mendefinisikan bagaimana kualitas sistem dalam melakukannya. Keduanya sama pentingnya — mengabaikan NFR sering menyebabkan kegagalan sistem di lingkungan produksi nyata.
 - **Use case** adalah teknik pemodelan yang mendeskripsikan interaksi antara aktor dan sistem untuk mencapai tujuan tertentu. Use case terdiri dari aktor, nama use case, prasyarat, alur normal, alur alternatif, alur kesalahan, dan pasca-kondisi.
 - **Use case diagram** menggunakan notasi UML standar untuk memvisualisasikan seluruh use case dan aktor beserta hubungannya (asosiasi, <<include>>, <<extend>>, generalisasi) dalam satu gambaran yang komprehensif.
 - **Studi kasus SafeHome** mendemonstrasikan bagaimana semua konsep di atas diterapkan dalam skenario nyata: dari wawancara elicitation hingga use case diagram dan tabel spesifikasi kebutuhan.
-